

Speculative MoE: Communication Efficient Parallel MoE Inference with Speculative Token and Expert Pre-scheduling

Yan Li^{*1} Pengfei Zheng^{*1} Shuang Chen¹ Zewei Xu¹ Yuanhao Lai¹ Yunfei Du¹ Zhengang Wang¹

Abstract

MoE (Mixture of Experts) prevails as a neural architecture that can scale modern transformer-based LLMs (Large Language Models) to unprecedented scales. Nevertheless, large MoEs' great demands of computing power, memory capacity and memory bandwidth make scalable serving a fundamental challenge and efficient parallel inference has become a requisite to attain adequate throughput under latency constraints. DeepSpeed-MoE, one state-of-the-art MoE inference framework, adopts a 3D-parallel paradigm including EP (Expert Parallelism), TP (Tensor Parallel) and DP (Data Parallelism). However, our analysis shows DeepSpeed-MoE's inference efficiency is largely bottlenecked by EP, which is implemented with costly all-to-all collectives to route token activation. Our work aims to boost DeepSpeed-MoE by strategically reducing EP's communication overhead with a technique named Speculative MoE. Speculative MoE has two speculative parallelization schemes, speculative token shuffling and speculative expert grouping, which predict outstanding tokens' expert routing paths and pre-schedule tokens and experts across devices to losslessly trim EP's communication volume. Besides DeepSpeed-MoE, We also build Speculative MoE into a prevailing MoE inference engine SGLang. Experiments show Speculative MoE can significantly boost state-of-the-art MoE inference frameworks on fast homogeneous and slow heterogeneous interconnects.

1. Introduction

The democratization of LLM (Large Language Model) has constantly been driven by the growing of model sizes. During the past five years, the largest trained LLM has increased its number of parameters by three orders of magnitude, and the scalability and economicity of training and inferring

massive LLMs now significantly challenge modern AI supercomputing hardware. To address the problem, the Mixture of Experts (MoE) (Fedus et al., 2022; Artetxe et al., 2022; Jiang et al., 2024b) models that sparsely activates one or more expert sub-networks for each input, is developed. Compared with their dense counterparts, MoE models parsimoniously train tens of trillions of parameters with uncompromised model accuracy, while keeping a sub-linear increase in computational costs as model size scales. MoE has prevailed and dominated in recent announcements and releases of industrial-strength LLMs including Gemini 1.5 (Team, 2024), Mixtral-8x7B/Mixtral-8x22B (Jiang et al., 2024a), DBRX (Mosaic-Research), Arctic (arc) DeepSeek-MoE-16B (Dai et al., 2024), DeepSeek V2 (DeepSeek-AI, 2024a), V3 (DeepSeek-AI, 2024b), R1 (DeepSeek-AI, 2025), Grok-1 (X-AI), QWen-MoE (Qwen-Team, 2024), etc.

Nevertheless, at inference time, massive MoEs models' voluminous expert and attention blocks still require huge amounts of GPU/NPU¹ cores, memory and bandwidth to compute, stash and load expert parameters for the forward pass. Existing MoE frameworks achieve inference scalability and performance with distributed parallel inference that spans interconnected GPU devices. To achieve adequate throughput under stringent latency requirements, an efficient multi-dimensional parallelization scheme that well partitions input tokens and expert and attention parameters, maximally utilizes and collates the computing and memory resources, and minimally imposes network communication overhead, is of pivotal importance.

To advance parallel MoE inference, DeepSpeed-MoE (Rajbhandari et al., 2022; Singh et al., 2023; Hwang et al., 2022), a state-of-the-art framework, adopts classic TP (Tensor Parallelism) and DP (Data Parallelism) and to parallelize the dense MHA (Multi Head Attention) layers and adopts a new model-parallel scheme named EP (Expert Parallelism) to parallelize the sparse expert layers. EP partitions and computes experts in parallel across GPUs, while notorious for incurring heavy communication overheads; the intermediate activation for individual tokens needs to be dispatched from the gating (e.g., Top-K gating) module on an arbitrary GPU *src* to its routed experts on GPU *dst*₁, ..., *dst*_K; af-

^{*}Equal contribution ¹Huawei Technology, China. Correspondence to: Pengfei Zheng <zhengpengfei18@huawei.com>.

¹We use GPU and NPU in this paper.

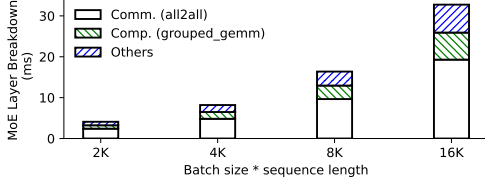


Figure 1: Latency breakdown for DeepSpeed-MoE inference over a single MoE layer. Hardware: 8-GPU (96GB) server with fast inter-GPU network (900GB/s); Model: DeepSeek-V2 236B; Dataset: LongBench; batch size * sequence length: 2K-16K.

ter expert FFN computation, token activation needs to be re-dispatched back to the source GPU *src* to merge different experts’ outputs, and to collocate with the residing KV cache for subsequent MHA computation. Collectively, dispatching and re-dispatching all outstanding tokens within and across devices can induce cluster-wide any-to-any shuffles, and DeepSpeed-MoE adopts two high-performance `all2all` collectives (e.g., NCCL PXN AllToAll) to accelerate such heavy communication processes.

Notwithstanding, our analysis indicates DeepSpeed-MoE’s inference performance remains severely bottlenecked EP’s costly `all2all` collectives; a preliminary experiment of DeepSpeed-MoE on serving DeepSeekV2-236B on 8 GPUs shows EP’s communication overhead accounts for $\sim 59.2\%$ and $\sim 47.1\%$ of the forward-pass latency of expert layers and entire model, though on fastest high-speed interconnects (cf., Figure 1). Such bottleneck can be severer when EP’s global shuffle goes through slower interconnects such as PCI-e and Ethernet. Thus, systematically reducing the communication cost of EP has become a top priority task to boost the efficiency and scalability of MoE inference.

In this study, we show the communication overhead of DeepSpeed-MoE’s parallel inference paradigm (EP+TP) can be losslessly reduced by recomposing its static deterministic parallelization scheme into a new, dynamic speculative scheme, which probes and forecasts the expert routing (or activation) paths for individual outstanding tokens and proactively co-shuffle tokens and experts to trim the excess communication of EP and TP. We name our speculative inference framework Speculative MoE (s-MoE); s-MoE has two speculative mechanisms, Speculative Token Reshuffling (s-TS) and Speculative Expert Pre-grouping (s-EG).

First, online, s-TS pre-shuffles outstanding tokens’ partial activation across GPU devices early amid TP and tries to collocate tokens’ activation with their predicted experts to route later in EP. Specifically, s-TS reformulates the `allreduce` collective of DeepSpeed-MoE’s TP into a new communication kernel named `shuffled-reduce-scatter`, which seamlessly merges the speculative token shuffling

operation with a `reduce-scatter` collective, and in addition, can eliminate one costly excess `allgather`. However, s-TS’s opportunistic trimming fails to save communication when expert activation paths are erroneously predicted, and when the routed experts (for individual tokens) are dispersed across many different GPU devices and servers, which can be true for MoE models with a large number of small experts. s-TS trains accurate probabilistic models with extensive preparatory tests to address the former problem, while relying on expert pre-grouping to tackle the latter problem. Second, offline, to reduce the likelihood of expert dispersion, s-EG pre-clusters experts that are likely to be activated by the same token or a clique of semantically related tokens onto the same device or server, with the predicted token-expert routes. We execute expert pre-grouping periodically offline since clustering and dispatching experts require solving sophisticated optimization and can incur overwhelming amounts of parameter exchange over the network, both of which are untimely for real-time inference. We list Speculative MoE’s contributions as below.

1. The inference throughput of s-MoE’s speculative token shuffling outperforms that of DeepSpeed-MoE by 1.58x-2.34x, 1.04x-2.34x, and 1.37x-5.98x, for TTFT, TPOT, and p90-TBT latency constraints, on different datasets, models and hardware.
2. Speculative expert pre-grouping (s-EG) on average can further boost s-TS by 27% in inference throughput. s-MoE (s-TS and s-EG together) achieves an end-to-end throughput that is 1.69x-2.37x, 1.14x-2.93x and 1.48x-6.54x higher than that of DeepSpeed-MoE, for TTFT, TPOT and p90-TBT constraints.
3. s-MoE’s implementation in another prevailing LLM serving engine SGLang (Zheng et al., 2024) boosts its inference throughput by 1.68x, 1.96x, and 1.97x, for TTFT, TPOT and p90-TBT constraints. SGLang features different MoE optimizations compared to DeepSpeed-MoE (EP+TP) concerning fused kernels and parallelization (TP). and the results show s-MoE as a generic optimization technique.
4. S-MoE features engineering optimizations that include high-performance kernel implementation, communication coalescing, dispatching deduplication, cached lookup table, etc. These optimizations together translate into an extra $\sim 7\%$ performance improvement.
5. We show that most tokens’ routed experts can be accurately predicted from the token itself and its few preceding tokens. s-MoE’s probabilistic models on average achieve 89% accuracy in speculating token-expert routes. Moreover, s-MoE features a balanced co-clustering algorithm to solve scheduling hints, which

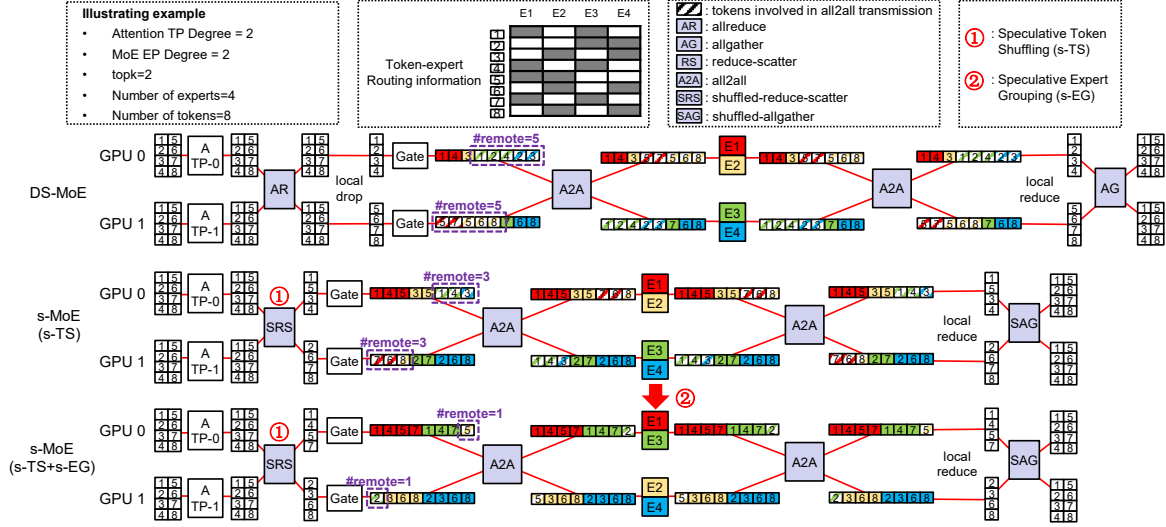


Figure 2: Example of s-MoE. Compared with DS-MoE (5 tokens in EP shuffling), speculative token shuffling (s-TS) replaces DS-MoE’s allreduce with a customized shuffled-reduce-scatter, which reduces EP’ shuffling into 3 tokens by pre-collocating tokens with their speculated experts to be route. Furthermore, speculative expert shuffling (s-EG) co-groups semantically similar experts and avoids tokens’ dispersed activation, further reducing EP shuffling into 1 token.

outperforms baselines using 2D Balanced K-Means and METIS (Karypis & Kumar, 1997).

2. Background

MoE Training. There has been extensive research on optimizing systems of MoE training systems, including FastMoE (He et al., 2021), FasterMoE (He et al., 2022), TAMoE (Chen et al., 2022), SmartMoE (Zhai et al., 2023), and FlexMoE (Nie et al., 2023). However such optimizations can not directly translate to inference scenarios as inference strongly emphasizes latency over throughput.

MoE Inference. Integrated serving engines such as DeepSpeed-MII (Holmes et al., 2024), TensorRT-LLM (NVIDIA)/vLLM (Kwon et al., 2023) have holistic optimization for LLM inference that spans serving schedulers (e.g., continuous batching), dedicated high-performance kernels, efficient parallelization, quantization, and elaborate compiler passes for graph-level optimizations. Built upon these general holistic optimizations for LLM inference, DeepSpeed-MoE (Rajbhandari et al., 2022) and its variants DeepSpeed-TED (Singh et al., 2023) and Tutel (Hwang et al., 2022) specifically optimize MoE models’ computation and communication. Another serving engine that explicitly declares optimization for MoE models is SGLang (Zheng et al., 2024). s-MoE specializes in optimizing MoE parallelization (particularly EP) and inherits wholistic optimizations from prior work.

Dynamic Mechanisms for MoE Inference - dynamic load balancing, dynamic backend switching. Lina (Li et al., 2023) probes the variation of expert hotness and allots non-

uniform expert replicas to achieve load-balanced expert computation. Similar studies (Huang et al., 2023) exist to pursue expert load balancing and mitigate other sources of MoE computing inefficiencies. EPS-MoE (Qian et al., 2025) optimizes the computation of MoE FeedForward Network (FFN) modules by dynamically selecting the best backend implementation of GroupGemm and DenseGemm.

Speculative MoE Inference - offloading and prefetching. Existing work on speculative MoE inference primarily focuses on prefetching offloaded experts and strategically saving GPU memories (Yi et al., 2023; Xue et al., 2024; Zhong et al., 2024), though offloading can extend inference latency and is rarely used in latency-critical serving scenarios. In contrast, s-MoE focuses on the speculative reduction of communication overheads and exposes no risks to compromise latency. The work also constructs probabilistic models to predict the token-expert routing paths, while s-MoE’s features modeling more comprehensive MoE information, i.e., intra-layer and inter-layer expert affinity and token-expert affinity, compared to prior work. Pre-gated MoE (Hwang et al., 2024) modifies the MoE model architecture to predict the experts to route at the next layer. s-MoE requires no modification to MoE architecture.

Speculative MoE Inference - communication reduction. ExFlow (Yao et al., 2024) is one MoE optimization most similar to s-MoE, which exploits the affinity between experts across adjacent layers to reduce remote routing and collocate closely related experts. ExFlow resembles s-MoE’s speculative expert pre-grouping but only incorporates inter-layer affinity between experts and lacks intra-layer affinity

between experts and the conjugate affinity between tokens and experts, while s-MoE leverages all three to co-schedule tokens and experts. Also, ExFlow can not handle the dispersed expert issue as it only considers Top-1 MoE gating.

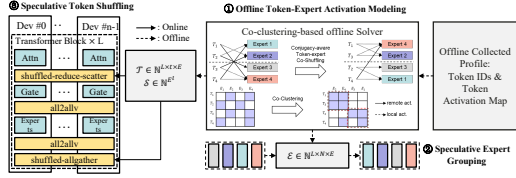


Figure 3: s-MoE workflow

3. Methodology

3.1. Speculative MoE (s-MoE): Overview

Figure 3 shows s-MoE’s workflow, in which the bold lines and dashed lines indicate online and offline operations. First, s-MoE collects token activation profiles, including token IDs and token-expert activation frequencies (① in Figure 3). s-MoE probabilistically models the token-expert routing likelihood and solves a balanced token-expert co-clustering problem to make the co-scheduling hints, which includes lightweight lookup tables (token-to-expert-cluster table $\mathcal{T}$, and expert-cluster-sequence-to-expert-cluster table $\mathcal{S}$) and experts grouping tables $\mathcal{E}$. Tables $\mathcal{T}$ and $\mathcal{S}$ are used by the shuffled-reduce-scatter (SRS) and shuffled-allgather (SAG) kernel (i.e., speculative token shuffling, s-TS, ② in Figure 3). $\mathcal{E}$ is used for speculative expert shuffling, s-EG (③ in Figure 3).

Figure 2 depicts an illustrating example of how s-MoE works. In the baseline DS-MoE (the first row), tokens are synchronized by an allreduce operator and partially dropped before the gate module. After being decided which expert to activate, the tokens are sent to their corresponding devices for expert computation. Then the tokens are sent back, reduced locally, and gathered. After applying s-TS (the second row), the experts each token is routed to are predicted in advance before the gate module. Then the tokens are shuffled and scattered to their affiliative expert groups via the SRS operator (① in Figure 2), reducing the all2all communication volume via increasing local activation rate. Furthermore, when applying s-EG (the third row), experts are co-shuffled with tokens offline (② in Figure 2), enabling s-TS to achieve even higher local activation rate. In the example, the number of remote-activated tokens are trimmed from 5 to 3 to 1 after adopting s-TS and s-EG. We detail the performance analysis and algorithms of s-TS and s-EG in the following subsections.

3.2. Performance Analysis - s-TS and s-EG

We derive the theoretical communication-saving volume from switching to the dynamic speculative schemes s-TS and s-EG as follows. We use the symbol $\nu_{[...]}$ to represent

the communication volume under hyper-parameters [...].

Preliminary1: communication volume of reduction collectives. Suppose there are G devices, the global batch size and the sequence are B and S . The communication volume of allreduce, allgather, reduce-scatter is given by $\nu_{G,B,S}(\text{AR}) = \frac{2(G-1)BS}{G}$, $\nu_{G,B,S}(\text{AG}) = \frac{(G-1)BS}{G}$, $\nu_{G,B,S}(\text{RS}) = \frac{(G-1)BS}{G}$.

Preliminary2: communication volume of the p2p collective. We define the percentage of tokens processed by local experts as the *local activation rate* (α), and the number of experts each token is routed to is k . The communication volume of all2all is given by $\nu_{G,B,S,k,\alpha}(\text{A2A}) = \frac{\alpha k BS}{G}$.

Figure 2 compares the s-MoE with DS-MoE. As we focus on the communication volume of attention TP and MoE EP. We implicitly suppose the DP degree of attention and the TP degree of MoE is 1. The communication pipeline of DS-MoE is $\text{AR} \rightarrow \text{A2A} \rightarrow \text{A2A} \rightarrow \text{AG}$. We suppose the averaged local activation rate of the A2A in DS-MoE is $\frac{1}{G}$. Then the communication volume of DS-MoE-EP is given by: $\nu_{G,B,S}(\text{AR}) + 2\nu_{G,B,S,k,\frac{1}{G}}(\text{A2A}) + \nu_{G,B,S}(\text{AG}) = \frac{3BS(G-1)}{G} + \frac{2BSk(G-1)}{G^2} = BS(3 - \frac{1}{G} - \frac{2}{G^2})$.

For s-MoE, the communication pipeline is $\text{SRS} \rightarrow \text{A2A} \rightarrow \text{A2A}$, and the communication volume is given by: $\nu_{G,B,S}(\text{RS}) + 2\nu_{G,B,S,k,\alpha}(\text{A2A}) = \frac{BS(G-1)}{G} + \frac{2BSk(1-\alpha)}{G} = BS(1 + \frac{2k(1-\alpha)-1}{G})$.

It can be derived that, the communication-saving volume of s-MoE compared with DS-MoE-EP and SGLang-EP are $BS(2 - \frac{2k(1-\alpha)+\frac{2}{G}}{G})$ and $BS(3 - \frac{2k(1-\alpha)+3}{G})$. With local activation rate, α , ranging from 0 to 1, the communication-saving volume ratio ranges from 32%~75% and 16%~69%.

Therefore, a high local activation rate α can trim the communication overhead greatly, which can be achieved via properly co-clustering tokens and experts to the same device. Our proposed s-MoE implements such an idea by speculatively shuffling of tokens on the fly by s-TS, together with the offline pre-grouping of experts from s-EG, to realize a higher local activation rate, as detailed next.

3.3. Predicting Expert Routing Path

In general, the routing of token x_j to k experts in the L^{th} MOE layer is governed by a gating network, expressed as

$$G_L(h_{L,j}) = \text{top-k}(\text{softmax}(\mathbf{W}_{L,g}h_{L,j} + \mathbf{b}_{L,g})),$$

where $h_{L,j}$ is the semantic representation of token x_j processed by earlier layers along with its preceding tokens (context), $\mathbf{W}_{L,g}$ and $\mathbf{b}_{L,g}$ are learnable parameters, and the top-k operation selects the K most relevant experts. This formulation indicates that the selection of experts at layer L is determined by both the token x_j and its context, leading to

an inherent **intra-layer token-expert affinity**. Furthermore, the expert choices for x_j in earlier layers are also determined by x_j and its context, which subsequently influences the input to deeper layers. Consequently, the expert choices at the L^{th} layer exhibit an association with those in preceding layers, namely, **inter-layer expert-expert affinity**.

Simplified tabularized conditional probabilistic model for the gating network to forecast expert routing paths.

We argue that a simple conditional probability model may suffice to predict the expert activation path of tokens implied by the above intra-layer/inter-layer affinity. We investigate both affinity phenomena empirically by conducting intermediate expert activation profiling experiments in the DeepSeek-MoE-16B and Mixtral-8x7B using the *Long-Bench* dataset, as shown by Figure 8 of Appendix. We find that semantically similar tokens are likely to activate a certain sub-group of experts, regardless of the context. Moreover, when tokens choose some concrete experts at certain layers, they tend to choose a rather fixed set of experts at the next layer with high probability.

Therefore, we simplify the **modeling of intra-layer token-expert affinity**, using only the token x_j to predict the probability of routing to k -th expert $E_k^{(L)}$ at the L -th MOE layer, $\Pr(E_k^{(L)}|x_j) = \mathbf{T}_{j,k}^{(L)} / \sum_{k=1}^{N^{(L)}} \mathbf{T}_{j,k}^{(L)}$, where $\mathbf{T}_{j,k}^{(L)}$ denotes the number of times the token x_j is routed to the k -th expert $E_k^{(L)}$ within L -th MOE layer in the dataset collected during the offline profiling stage, for $j = 1, \dots, t$ and $k = 1, \dots, N^{(L)}$. For fast lookup of the matched expert for a given token, we tabularize the above token-expert relationship by the **token-2-expert confidence table** $\mathcal{C}_p \in \mathbb{N}^{t \times N}$, where $\mathcal{C}_{p,jk} = \Pr(E_k|x_j)$ and we omit the layer index for representing the case in an arbitrary layer. For an out-of-vocabulary (OOV) token $\tilde{x}$ not covered by the profile, we use the expert activation probabilities of the nearest known token x' in the embedding space, measured by the cosine distance, as the prediction, which ensures robust predictions even for OOV tokens.

To simplify **modeling inter-layer expert-expert affinity**, because we only care about the device-level token shuffling, we model the coarser activation expert cluster sequence rather than the activation expert sequence with a n -gram model $\Pr(D_k^{(L)}|D^{(L-1)}, \dots, D_{(L-n)})$ where $D_{(l)} \in \{1, \dots, Q\}$ denote the device index where the chosen expert locates at the l -th layer for an arbitrary token. We also tabularize the above expert-expert relationship via a expert-cluster-sequence-2-expert-cluster confidence table $\mathcal{A}_p \in \mathbb{N}^{Q^n \times Q}$ and the corresponding expert-cluster-sequence-2-expert-cluster table $\mathcal{A} = \text{argmax}(\mathcal{A}_p, \text{axis} = 1) \in \mathbb{N}^{Q^n \times 1}$. We use 2-gram in practice. Together, the expert routing paths can be predicted accurately by either the intra-layer token-expert model or inter-layer expert-expert model.

3.4. Balanced Token-Expert Co-clustering

We illustrate how such expert-routing forecasting models can guide the co-dispatching of tokens and experts. Different from existing solvers such as Metis and two-stage Kmeans, s-MoE models the token-expert co-clustering problem as a 0 - 1 integer programming (ILP) problem.

From the offline profiling, we obtain the number of deduplicated (un-deduplicated) tokens t (S), the number of experts per layer N , the number of clusters E (also EP degree), the token j frequency $\mathbf{a}_j$, and the predicted probability $\mathcal{C}_{p,jk}$ that token j activates expert k . The decision integer variables are set as the placement $\mathbf{R}_{ij} \in \{0, 1\}$ of token j to cluster i and the placement $\mathbf{C}_{ij} \in \{0, 1\}$ of expert j to cluster i . We aim to minimize an objective function $\mathcal{L} = \theta \sum_{i=1}^E \left| \sum_{j=1}^t (\mathbf{R}_{ij} \mathbf{a}_j) - \frac{S}{E} \right| + (1 - \theta) \sum_{i_1 \neq i_2} \left(\sum_{j=1}^t \sum_{k=1}^N (\mathbf{R}_{i_1 j} \mathbf{C}_{i_2 k} \mathcal{C}_{p,jk} \mathbf{a}_j) \right)$, where the left part is to ensure that the token frequencies of different clusters as even as possible to promote load balancing among EP ranks, and the right part is to minimize the `all2all` communication overhead caused by remote activation (i.e., the summation of all the activations of tokens and experts belonging to different clusters), a factor $\theta \in (0, 1)$ controlling the percentage of two sub-objectives. We further require that each token belongs to only one class, each expert belongs to only one class, and the number of experts in each class is equal by adding hard constraints $\sum_{i=1}^E \mathbf{R}_{ij} = 1$, for $j = 1 \dots t$, $\sum_{i=1}^E \mathbf{C}_{ij} = 1$, for $j = 1 \dots N$, and $\sum_{j=1}^N \mathbf{C}_{ij} = \frac{N}{E}$, $i = 1 \dots t$.

The above ILP problem is difficult to solve directly using LP solvers, given a large number of intermediate variables introduced in the linearization process. s-MoE provides a two-phase cross-entropy optimization algorithm. It can quickly obtain a feasible solution while ensuring load balancing. The detailed co-clustering algorithm can be referred to in § B in the Appendix. The solution can then be applied to online speculative token shuffling and offline speculative expert grouping, which can be referred to in § B as well.

4. Implementation and System Optimization

s-MoE is implemented in ~ 5000 LOC Python code with Triton (OpenAI)-implemented kernels. The current implementation of s-MoE is a featured plugin for community-leading open-sourced MoE inference frameworks DeepSpeed-MoE (Rajbhandari et al., 2022) and SGLang (Zheng et al., 2024), and future support for other prevailing frameworks including but not limited to vLLM (Kwon et al., 2023) and TensorRT-LLM (NVIDIA) is planned. Note that besides the methodological innovation made by s-TS and s-EG, s-MoE also features the following system-level optimizations. (a) **Optimized Triton shuf-**

Server	Configuration
GPU-Server-A	CPU: Two 44-core Intel 8458P CPUs
	CPU RAM: 2TB DDR5
	GPU: 8 cards, GPU RAM: 96GB HBM per card
	Inter-GPU: 900GBps high-speed link
GPU-Server-B	CPU: Two 44-core Intel 8458P CPUs
	CPU RAM: 1TB DDR5
	GPU: 8 cards, GPU RAM: 48GB GDDR6 per card
	Inter-GPU: 2*PCIe switch (64GBps), UPI (20GBps)

Table 1: Experiment hardware configuration

fling kernel. To enable efficient s-TS, we implement fused Triton kernels `shuffled-reduce-scatter` (SRS) and `shuffled-allgather` (SAG), which are fused with conventional `reduce-scatter` and `allgather` collectives, respectively. SRS and SAG compute shuffles’ source and destination information with an optimized `argsort` kernel (outperforming Pytorch-native version by 25%) and seamlessly embed the information into the ring-based implementation of native `reduce-scatter` and `allgather` with negligible (1%) overheads. **(b) Efficient EP with one-shot `all2allv`.** EP can be implemented with regular `all2all` or irregular `all2allv`. The existing EP implementations with regular `all2all` send fixed-shaped padded tensors to experts with no need to send additional metadata (e.g., indices of selected experts) but add additional communication costs due to padding. Other EP implementations improve `all2all` with irregular `all2allv`, but need to launch two separate collectives, one for metadata (e.g., expert indices and gating scores) and one for MHA activation. s-MoE further improves this by consolidating MHA activation and metadata into a compact communication packet and initiates only a one-shot `all2allv` to avoid additional kernel-launch overheads. **(c) De-duplication for EP.** When multiple experts activated by the same token are located on the same GPU, there is no need to send two copies of the token’s MHA activation to that device. But existing studies such as DeepSpeed-MoE and Tutel (Hwang et al., 2022) ignore this and send redundant data that further burdens EP. s-MoE implements a de-duplication kernel into EP to detect such compaction opportunities, and further reduces EP’s communication.

5. Experiment

5.1. Experimental Environments

We evaluate Speculative MoE on two 8-GPU servers with different interconnect speeds and topologies (cf., Table 1). **(a) GPU-Server-A: fast homogeneous interconnects.** GPU-Server-A represents commodity GPU servers unified for both training and inference, which are configured with 96GB-HBM per GPU and fast homogeneous interconnects. GPUs inside a server can communicate with each other at a premium bandwidth (900GBps). **(b) GPU-Server-B: slow heterogeneous interconnects.** GPU-Server-A represents commodity GPU servers dedicated to economic in-

ference, which have no HBM and no high-speed NVLink interconnects. Each server has eight GPUs (48GB GDDR6); the left group of four GPUs is interconnected via a PCIe-5.0 switch (63.01GBps) and the right group of four GPUs holds the same symmetric configuration. Any GPU in one group communicating with another GPU in the other group needs to pass through a slow indirect link, i.e., CPUs’ UPI (Ultra Path Interconnect) interface (20.8GBps).

5.2. Models, Datasets and Workload Traces

Model. We choose two types of typical MoE models for evaluation, i.e., MoE models with a *large number of small experts*, represented by DeepSeek-V2, and MoE models with a *small number of large experts*, represented by Mixtral-8x7B. Both DeepSeek-V2 and Mixtral-8x7B are prevailing open-sourced MoE models.

Dataset. We use the following three representative datasets *LongBench* (Bai et al., 2024), *GSM8K* (Grade School Math 8K) (Cobbe et al., 2021) and *HumanEval* (Chen et al., 2021).

Workload. It is important that workload characteristics, especially the length of input (i.e., prefilling) and output (i.e., decoding) per user request, should reflect the characteristics of real requests in production settings. We draw real requests’ input and output length from the Mooncake production trace (Qin et al., 2024) and the Azure LLM Inference Trace (Stojkovic et al., 2024). To synthesize a request, we first randomly sample a pair of input and out length from the MoonCake or the Azure trace with equal probability, and secondly, sample an entry from LongBench, GSM8K, or HumanEval with equal probability to realize the entry’s prefilling and decoding (up to the sampled length).

5.3. Baselines and Performance Metrics

We build s-MoE into two state-of-the-art MoE serving frameworks, each with differently implemented inference optimizations spanning parallelization schemes, high-performance fused kernels, quantization, etc.

TP-EP Baseline - DeepSpeed-MoE, DeepSpeed-TED, and Tutel: DeepSpeed-MoE (Rajbhandari et al., 2022) is a strong inference framework known for efficient scalable MoE computation and communication. DeepSpeed-TED (Singh et al., 2023) is an optimized version of DeepSpeed-MoE that further reduces communication overhead in DeepSpeed-MoE. Tutel (Hwang et al., 2022) is a customized version of DeepSpeed-MoE that optimizes both MoE training and inference with high-performance fused kernels. We choose an integrated version of DeepSpeed-MoE that includes all optimizations from DeepSpeed-TED and Tutel, as a state-of-the-art baseline, the baseline adopts TP for attention layers and EP for expert layers.

TP Baseline - SGLang: SGLang (Zheng et al., 2024) is a

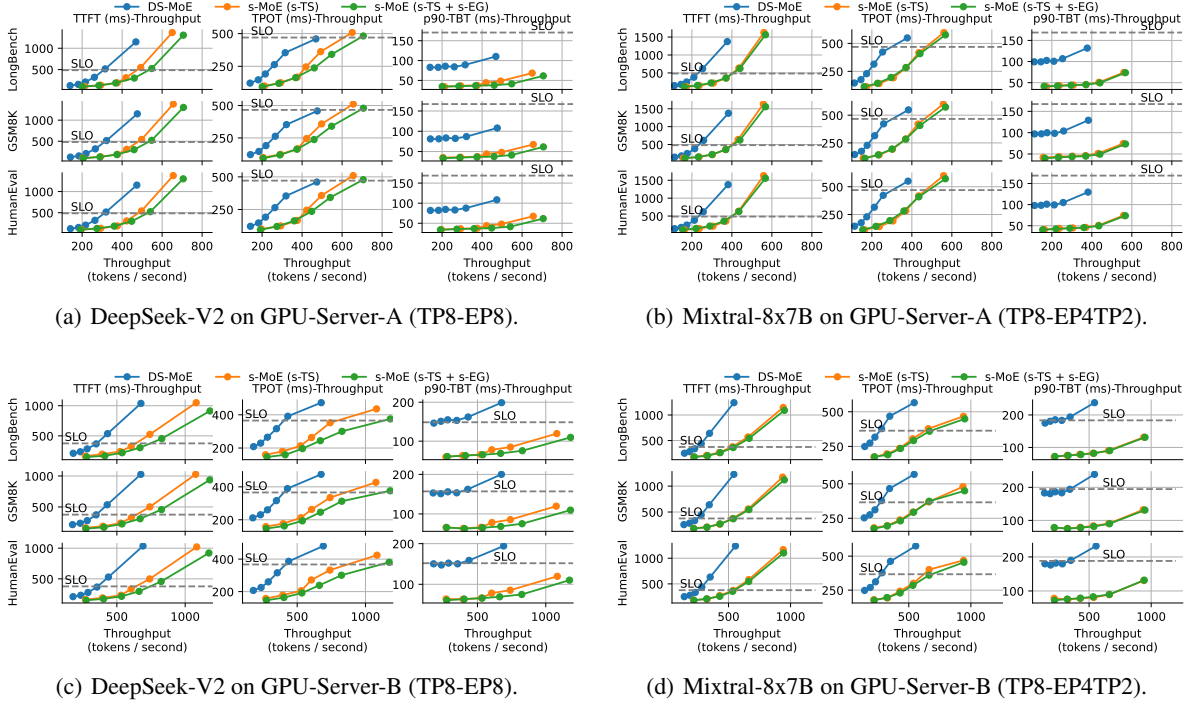


Figure 4: Inference throughput under TTFT, TPOT and p90-TBT latency constraints. DS-MoE: DeepSpeed-MoE. s-MoE: Speculative MoE, s-TS: Speculative Token Shuffling, s-EG: Speculative Expert Grouping. Note we tune parallel configurations and report the one DeepSpeed-MoE performs best.

prevailing open-source LLM inference framework, adopting a bunch of optimization including but not limited to continuous batching, paged attention, flash attention, radix-attention, advanced quantization, etc. SGLang declares optimizations for MoE models with high-performance, fused triton kernels (OpenAI) to implement TP-based parallelization for both attention and expert layers.

Metrics: **Throughput** is the number of tokens (tokens/s) that an inference system can process under a time limit. We report the average throughput over a batch of prompts under the following latency constraints. **TTFT (Time to First Token)** measures the duration between the request’s arrival and the first token’s generation time. **TPOT (Time per Output Token)** is the average time to generate an output token in the decoding phase, i.e., the total decoding time of a request normalized by the number of tokens generated. **TBT (Time between Tokens)** is the latency of every subsequent token generated in the decoding phase. We report the 90th percentile of TBT (p90 TBT). Following prior work (Wu et al., 2024b;a), we set the latency SLO as $5 \times$ of the latency (i.e., TTFT, TPOT, or p90 TBT) under the lightest input load (minimal input length times batch size sampled from datasets and trace). For each dataset, we use 20% of the data to train the token activation prediction model, and the left 80% is used to sample experimental requests.

5.4. Evaluation - Speculative MoE vs. DeepSpeed-MoE

Figure 4 shows the end-to-end performance evaluation of Speculative MoE (s-MoE) and DeepSpeed-MoE (DS-MoE). We evaluate two versions of s-MoE, namely s-MoE w/t s-TS, and s-MoE w/t s-TS+s-EG, to separate the contributions of Speculative Token shuffling (s-TS) and Speculative Expert Pre-grouping (s-EG), respectively.

DeepSeek-V2. For DeepSeek-V2, s-MoE achieves 1.72x, 1.15x and 1.49x throughput improvements compared to DS-MoE on GPU-Server-A under TTFT, TPOT and p-90-TBT SLO constraints, respectively. Results show s-MoE outperforms DS-MoE even on the fastest (900GBps) homogeneous interconnects. Results show s-MoE can still benefit production settings configured with high-end GPU interconnects (e.g., NVLink or UALink), as hardware and technology advances alone can not completely remove the current communication bottleneck of MoE inference.

On GPU-Server-B, which is configured with slow heterogeneous interconnects (i.e., PCI-e and UPI), s-MoE attains a much larger boost over DS-MoE, where the throughput improvements rise to 1.89x, 2.35x, and 4.3x under the aforementioned SLOs. Results show that s-MoE is more productive and supportive for economic inference settings that lack homogeneous high-speed hardware interconnects, and

algorithm and software optimizations remain more effective and cost-efficient remedies. For DeepSeek-V2, separate analyses show that s-TS and s-EG on average contribute to 73% and 27% of the end-to-end throughput improvement of s-MoE, respectively. The results validate the effectiveness and essentialness of the two key designs within s-MoE.

Mixtral-8x7B. For Mixtral-8x7B, the throughput improvements of s-MoE compared to DS-MoE is marginally lower to that of DeepSeek-V2, i.e., 1.72x, 1.72x and 1.49x on GPU-Server-A and 2.34x, 2.70x and 2.92x on GPU-Server-B, under TTFT, TPOT and p90-TBT SLO constraints. The number of experts per layer of Mixtral-8x7B is smaller (only 8 experts per layer), and thus, the gated experts to route are more concentrated rather than dispersed, which results in lower effectiveness (only 4% on average) for s-EG. However, s-TS remains effective in boosting DS-MoE. Overall, the consistent win of s-MoE over DS-MoE on both DeepSeek-V2 featuring many small experts and Mixtral-8x7B featuring few big experts, and on both high-end and low-end interconnects substantiate s-MoE as a generic approach to boost modern MoE inference.

5.5. A Detailed Look at EP Communication Reduction

In Figure 5, we show the LAR (Local Activation Rate) and the resulting latency of a single expert layer on GPU-Server-A. Local activation means the tokens’ activation is routed to an expert collocated on the same GPU device, and thus remote routing and its associated EP communication can be skipped. A higher LAR is demanded. Results show, compared with DS-MoE, s-MoE can increase LAR by 43% and 61% for DeepSeek-V2 and Mixtral-8x7B, which translates to 40.4%/68.8% latency reduction of the belonging expert layer. Besides DS-MoE and s-MoE, other bars in the figure are measured by mocking the routing module of DS-MoE and skipping the delays in communication to fabricate hypothetical baselines just for reference. Note that a 100% LAR may not be achieved in theory as different tokens can contradict each other to group their own hot experts but GPU memory is limited.

5.6. Boosting SGLang with Speculative MoE

Besides DS-MoE, we build a version of s-MoE into another state-of-the-art, open-sourced LLM inference engine SGLang (Zheng et al., 2024). Figure 7 shows, over all three datasets, s-MoE on average boosts SGLang’s inference throughput by 1.68x, 1.96x, and 1.97x under TTFT, TPOT, and p90-TBT latency constraints, even over the fastest (900GBps) GPU interconnects. The results verify that s-MoE’s optimizations are not dedicated to DeepSpeed-MoE, and can serve as a generic booster for a spectrum of MoE and LLM inference engines.

5.7. Algorithm Evaluation

The token and expert co-shuffling algorithm needs to find balanced co-clusters of tokens and experts, with experts having a maximal likelihood of being gated (routed) from tokens within the same cluster, and minimal likelihood across clusters. An additional regularizer is load balancing that ensures hot and cold experts are relatively evenly distributed. s-MoE adopts a cross-entropy-optimization-based (CEO-based) algorithm to approximately solve the problem and is evaluated against two other baseline algorithms, METIS (Karypis & Kumar, 1997) and 2D Balanced KMeans, using the same predicted routing likelihood as affinity. METIS models the token-expert routing matrix as an undirected graph and solves the minimum k-cut solution. 2D Balanced KMeans performs a two-stage balanced KMeans clustering for tokens and experts with the predicted routing affinity separately. Figure 6 shows s-MoE achieves the best LAR (highest communication reduction) with balanced expert clusters; its LAR and imbalance rate outperforms the best baseline by 14.2% and 10.2%, respectively.

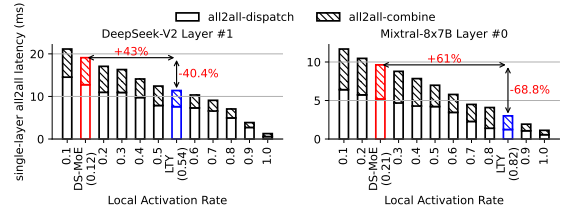


Figure 5: Local activation rate against overall EP overhead.

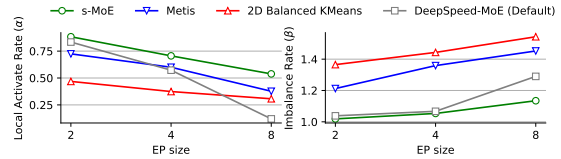


Figure 6: Local activation rate (α) and load-imbalance rate (β) of s-MoE-CEO and baseline algorithms.

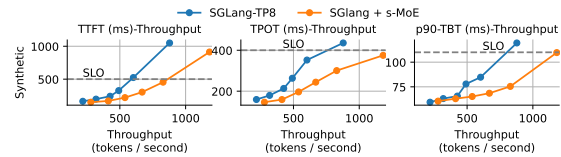


Figure 7: s-MoE boosting SGLang. Model: DeepSeek-V2. Dataset: Longbench, GSM8K and HumanEval. Server: GPU Server A. Parallelism: SGLang - Attention(TP8), Expert(TP8); s-MoE: Attention(TP8), Expert(EP8).

6. Conclusion

The communication overhead of expert parallelism renders a significant bottleneck in serving large-scale MoE models. We present s-MoE, which can proactively and losslessly trim EP’s all2all communication volume. Experiments

in prevailing MoE serving engines DeepSpeed-MoE and SGLang show that s-MoE can significantly reduce communication overhead and boost inference throughput under differently specified SLO constraints.

References

- Snowflakes-arctic. [Online]. Accessed 23 Oct 2024, <https://www.snowflake.com>.
- Artetxe, M., Bhosale, S., Goyal, N., Mihaylov, T., Ott, M., Shleifer, S., Lin, X. V., Du, J., Iyer, S., Pasunuru, R., Anantharaman, G., Li, X., Chen, S., Akin, H., Baines, M., Martin, L., Zhou, X., Koura, P. S., O’Horo, B., Wang, J., Zettlemoyer, L., Diab, M., Kozareva, Z., and Stoyanov, V. Efficient large scale language modeling with mixtures of experts, 2022. URL <https://arxiv.org/abs/2112.10684>.
- Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. Longbench: A bilingual, multitask benchmark for long context understanding, 2024. URL <https://arxiv.org/abs/2308.14508>.
- Chen, C., Li, M., Wu, Z., Yu, D., and Yang, C. Ta-moe: Topology-aware large scale mixture-of-expert training. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 22173–22186. Curran Associates, Inc., 2022.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. 2021.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dai, D., Deng, C., Zhao, C., Xu, R. X., Gao, H., Chen, D., Li, J., Zeng, W., Yu, X., Wu, Y., Xie, Z., Li, Y. K., Huang, P., Luo, F., Ruan, C., Sui, Z., and Liang, W. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models, 2024.
- DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024a.
- DeepSeek-AI. Deepseek-v3 technical report, 2024b. URL <https://arxiv.org/abs/2412.19437>.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL <http://jmlr.org/papers/v23/21-0998.html>.
- He, J., Qiu, J., Zeng, A., Yang, Z., Zhai, J., and Tang, J. Fastmoe: A fast mixture-of-expert training system, 2021. URL <https://arxiv.org/abs/2103.13262>.
- He, J., Zhai, J., Antunes, T., Wang, H., Luo, F., Shi, S., and Li, Q. Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPOPP ’22, pp. 120–134, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392044. doi: 10.1145/3503221.3508418. URL <https://doi.org/10.1145/3503221.3508418>.
- Holmes, C., Tanaka, M., Wyatt, M., Awan, A. A., Rasley, J., Rajbhandari, S., Aminabadi, R. Y., Qin, H., Bakhtiari, A., Kurilenko, L., and He, Y. DeepSpeed-fastgen: High-throughput text generation for llms via mii and DeepSpeed inference, 2024.
- Huang, H., Ardalani, N., Sun, A., Ke, L., Lee, H.-H. S., Sridhar, A., Bhosale, S., Wu, C.-J., and Lee, B. Towards moe deployment: Mitigating inefficiencies in mixture-of-expert (moe) inference, 2023. URL <https://arxiv.org/abs/2303.06182>.
- Hwang, C., Cui, W., Xiong, Y., Yang, Z., Liu, Z., Hu, H., Wang, Z., Salas, R., Jose, J., Ram, P., Chau, J., Cheng, P., Yang, F., Yang, M., and Xiong, Y. Tutel: Adaptive mixture-of-experts at scale. *CoRR*, abs/2206.03382, June 2022. URL <https://arxiv.org/pdf/2206.03382.pdf>.
- Hwang, R., Wei, J., Cao, S., Hwang, C., Tang, X., Cao, T., and Yang, M. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 1018–1031, 2024. doi: 10.1109/ISCA59077.2024.00078.

- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Hanna, E. B., Bressand, F., Lengyel, G., Bour, G., Lample, G., Lavaud, L. R., Saulnier, L., Lachaux, M.-A., Stock, P., Subramanian, S., Yang, S., Antoniak, S., Scao, T. L., Gervet, T., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mixtral of experts, 2024a.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Hanna, E. B., Bressand, F., Lengyel, G., Bour, G., Lample, G., Lavaud, L. R., Saulnier, L., Lachaux, M.-A., Stock, P., Subramanian, S., Yang, S., Antoniak, S., Scao, T. L., Gervet, T., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mixtral of experts, 2024b. URL <https://arxiv.org/abs/2401.04088>.
- Karypis, G. and Kumar, V. Metis: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices. 1997.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSPP ’23*, pp. 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Li, J., Jiang, Y., Zhu, Y., Wang, C., and Xu, H. Accelerating distributed MoE training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC ’23)*, pp. 945–959, Boston, MA, July 2023. USENIX Association. ISBN 978-1-939133-35-9. URL <https://www.usenix.org/conference/atc23/presentation/li-jiamin>.
- Mosaic-Research. Databricks-dbrx. [Online]. Accessed 23 Oct 2024, <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm>.
- Nie, X., Miao, X., Wang, Z., Yang, Z., Xue, J., Ma, L., Cao, G., and Cui, B. Flexmoe: Scaling large-scale sparse pre-trained model training via dynamic device placement. *Proc. ACM Manag. Data*, 1(1), May 2023. doi: 10.1145/3588964. URL <https://doi.org/10.1145/3588964>.
- NVIDIA. Tensorrt-llm. [Online]. Accessed 23 Oct 2024, <https://github.com/NVIDIA/TensorRT-LLM>.
- OpenAI. Applied ai experiments and examples for pytorch. [Online]. Accessed 23 Oct 2024, <https://github.com/pytorch-labs/applied-ai/tree/main>.
- Qian, Y., Li, F., Ji, X., Zhao, X., Tan, J., Zhang, K., and Cai, X. Eps-moe: Expert pipeline scheduler for cost-efficient moe inference, 2025. URL <https://arxiv.org/abs/2410.12247>.
- Qin, R., Li, Z., He, W., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: A kvcache-centric disaggregated architecture for llm serving, 2024. URL <https://arxiv.org/abs/2407.00079>.
- Qwen-Team. Qwen1.5-moe: Matching 7b model performance with 1/3 activated parameters”, February 2024. URL <https://qwenlm.github.io/blog/qwen-moe/>.
- Rajbhandari, S., Li, C., Yao, Z., Zhang, M., Aminabadi, R. Y., Awan, A. A., Rasley, J., and He, Y. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 18332–18346. PMLR, 17–23 Jul 2022. URL <https://proceedings.mlr.press/v162/rajbhandari22a.html>.
- Singh, S., Ruwase, O., Awan, A. A., Rajbhandari, S., He, Y., and Bhatele, A. A hybrid tensor-expert-data parallelism approach to optimize mixture-of-experts training. In *Proceedings of the 37th International Conference on Supercomputing, ICS ’23*, pp. 203–214, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700569. doi: 10.1145/3577193.3593704. URL <https://doi.org/10.1145/3577193.3593704>.
- Stojkovic, J., Zhang, C., Íñigo Goiri, Torrellas, J., and Choukse, E. Dynamollm: Designing llm inference clusters for performance and energy efficiency, 2024. URL <https://arxiv.org/abs/2408.00741>.
- Team, G. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024. URL <https://arxiv.org/abs/2403.05530>.
- Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., and Jin, X. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism, 2024a. URL <https://arxiv.org/abs/2404.09526>.
- Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., and Jin, X. Fast distributed inference

serving for large language models, 2024b. URL <https://arxiv.org/abs/2305.05920>.

X-AI. Xai-grok-1. [Online]. Accessed 23 Oct 2024, <https://github.com/xai-org/grok-1>.

Xue, L., Fu, Y., Lu, Z., Mai, L., and Marina, M. Moe-infinity: Offloading-efficient moe model serving, 2024. URL <https://arxiv.org/abs/2401.14361>.

Yao, J., Anthony, Q., Shafi, A., Subramoni, H., K., D., and Panda. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference, 2024.

Yi, R., Guo, L., Wei, S., Zhou, A., Wang, S., and Xu, M. Edgemoe: Fast on-device inference of moe-based large language models, 2023. URL <https://arxiv.org/abs/2308.14352>.

Zhai, M., He, J., Ma, Z., Zong, Z., Zhang, R., and Zhai, J. SmartMoE: Efficiently training Sparsely-Activated models through combining offline and online parallelization. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pp. 961–975, Boston, MA, July 2023. USENIX Association. ISBN 978-1-939133-35-9. URL <https://www.usenix.org/conference/atc23/presentation/zhai>.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: Efficient execution of structured language model programs, 2024. URL <https://arxiv.org/abs/2312.07104>.

Zhong, S., Liang, L., Wang, Y., Wang, R., Huang, R., and Li, M. Adapmoe: Adaptive sensitivity-based expert gating and management for efficient moe inference, 2024. URL <https://arxiv.org/abs/2408.10284>.

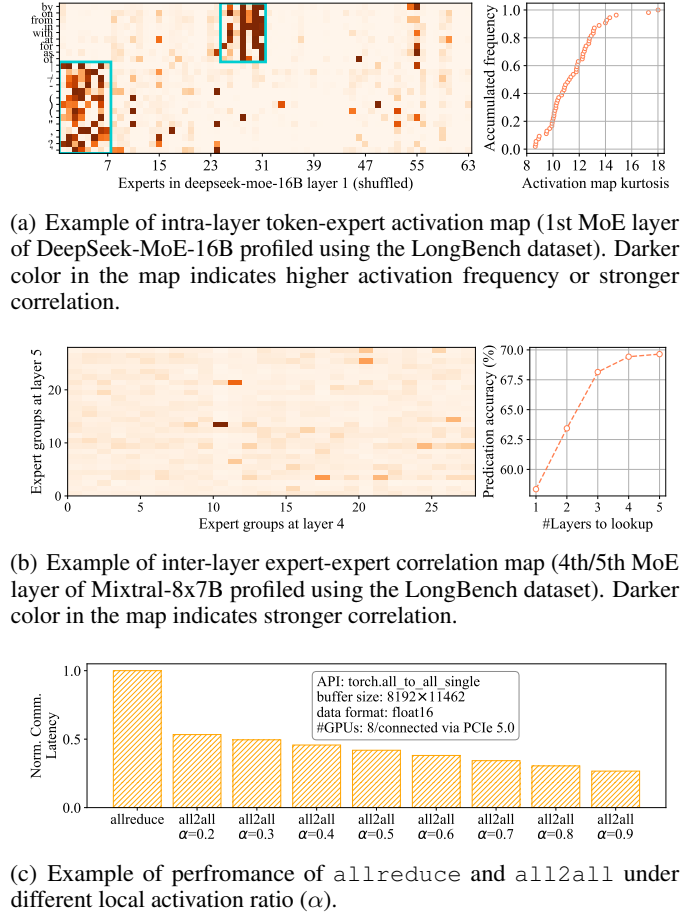


Figure 8: Conjugacy illustration and collective communication micro-benchmark.

A. Empirical Study of Token-expert Affinity

A.1. Intra-layer bi-clustered token-expert Conjugacy

Within each MoE layer, each expert module in an LLM layer is trained to process a particular semantic domain of tokens. Tokens and experts exhibit high affinity in different dimensions. We profile the intermediate activation of the gating module in each MoE layer in the DeepSeek-MoE-16B and Mixtral-8x7B. One important observation shows that strong bi-clustered conjugacy between tokens and experts, as shown in Figure 8. That is, experts are likely to be activated by a certain sub-group of tokens with high semantic affinity, while are not likely to be activated by other general tokens in vocabulary. And from tokens perspective, it is true that semantically similar tokens are likely to activate a certain sub-group of experts. We name this phenomenon bi-clustered conjugacy.

Inter-layer expert-expert affinity The left picture of Figure 8(b) shows the activation correlation of the 4th and 5th layer of the Mixtral-8x7B model. The x-/y-axis represents the expert groups of a layer. For Mixtral-8x7B, each token is routed 2 out of 8 experts at each layer. Thus, the number of expert groups at each layer is $\binom{8}{2} = 28$. When tokens choose some concrete experts at the fourth layer, they tend to choose a rather fixed set of experts at the next layer with high probability. We name this phenomenon *inter-layer expert-expert affinity*.

Simple conditional probability model for token activation path The above examples illustrate the tabularized relationship between tokens and experts. We argue that, simple conditional probability model can work to predict the activation

path of tokens by combining the above intra-layer conjugacy and/inter-layer affinity. We first calculate the kurtosis² of each token’s activation map. The right picture of Figure 8(a) shows most of the kurtosis values are higher than 8, indicating that the to-route experts for each token concentrate on a narrowed set, regardless of the context. We further use partial (25%) of the profile dataset to calculate each tokens’ most routed top-k experts. Then the static top-k experts are used to predict the tokens activation using the left part (75%) of the profile dataset, achieving a 96.3% precision and a 78.8% F1 score. The right part of Figure 8(b) predicts the next-layer-activation via looking back the prior layers. As we know more about the previous activation sequence at layer L , we can predict the activation at layer $L + 1$ with higher confidence (about 70% when looking back 5 layers).

Expert pre-grouping and token re-batching Leveraging the intrinsic conjugacy between tokens and experts in MoE models to achieve token-expert co-dispatching, may help reduce communication volume and boost distributed parallel inference. First, similar experts can be pre-grouped together at deployment time based on pre-profiled and modeled affinity. Second, on the fly, individual tokens can be re-shuffled and re-batched to the GPU devices that host the expert groups whose member experts have largest modeled conjugacy with the token. Such co-scheduling aims to avoid scattered, cross-device token-expert activations, or equivalently, maximize the probability of intro-device, local activations. Figure 8(c) shows a micro-benchmark experiment, testing the `all2all` latency under different local activation rate using the `nccl all2allv` API. With the local activation rate (α) varying from 0.2 to 0.9, the latency decreases gradually, showing the performance gains with improved expert-token co-scheduling.

B. Algorithm for the s-TS/s-EG Solver

The Rubinstein Cross-Entropy Optimization (CEO) algorithm is a Monte Carlo method used primarily for optimization and importance sampling. Similar to the evolutionary algorithm, the CEO sprinkles points in space according to a certain rule, obtains the error of each point, and then determines the rule of the next round of sprinkle points according to the error information. The cross entropy method is named because the objective of the method (theoretically) is to minimize the cross entropy between the data distribution obtained by random scattered points and the actual data distribution (equivalent to minimizing the KL distance), trying to make the sampling distribution (sprinkling points) the same as the real distribution.

Algorithm 1 describes the CEO-based token-expert co-scheduling algorithm in s-MoE. The algorithm models each expert/token’s choice of category as a D-dimensional multinomial distribution, and the probability of polynomial distribution is determined by p_matrix_ep and p_matrix_tk . Under the initial conditions, the probability of selecting each class by expert/token is set to equal. In each iteration, several class label samples are first sampled according to the probability matrix (lines 25-26). The sampling function `sample` samples label points from the D-dimensional polynomial distribution (line 11) and ensures that the number of experts in each category is equal (line 12-15). A relaxation factor β is used to ensure that the token frequency of each category is as equal as possible (lines 16-19). After the sampling is complete, the algorithm calculates the local activation frequency of each sample (line 27) as the score, and selects samples with top- $\rho \times 100\%$ scores to update the probability matrix (line 27–line 28). After completing the n_steps iteration, the algorithm takes the cluster with the highest probability for each expert/token as its cluster label (lines 32-33). The cluster label of expert/token actually represents the number of the device to which expert/token is scheduled. At the same time, the value of the probability matrix is used as the confidence that each token belongs to its cluster. In practice, the hyperparameters β and ρ are set as 1.1 and 0.2. By now, the token-device scheduling table $\mathcal{T}$, token-device scheduling confidence table $\mathcal{T}_p$, and expert-device scheduling table $\mathcal{E}$ are generated. After the scheduling table $\mathcal{E}$ is constructed, the experts at each layer need to be rearranged according to the scheduling table during online inference service deployment. In addition, the s-MoE rearranges the gating module by column to implement transparent expert shuffle. The semantics of other layers are not affected. The rearranged experts are highly boxed, so that the token activation at each layer is de-cohesive, and the redundant network communication overhead caused by dispersive activation is reduced.

B.1. Modeling Inter-layer Activation Conjugacy

Leveraging the conditional probability model described in § A.1, we use a simple probability-based first-order Markov chain to model the inter-layer activation conjugacy. To reduce the combination space, we model the activation device sequence rather than the activation expert sequence, because we only care about the device-level token rebatching. When looking

²Kurtosis is a measure of the tailedness of a distribution. High Kurtosis indicates a token favors several fixed experts during multiple occurrences.

Algorithm 1 CEO-based co-clustering algorithm

input: n_steps : number of iteration steps; $\mathcal{C}_p$: the token-2-expert confidence table; $\mathbf{a}$: the token frequency; K : number of samples of each sampling; N : number of experts per layer; E : number of co-clusters; t : number of tokens; f : token load-balance factor; ρ : percentage of tokens to choose during each iteration

output: $\mathcal{E}$: expert labels; $\mathcal{T}$: token labels; $\mathcal{T}_p$: confidence of tokens choosing specific experts

```

1   $p\_matrix\_ep \leftarrow \text{ones}(N, E) / E$ 
2   $p\_matrix\_tk \leftarrow \text{ones}(t, E) / E$ 
3  Function  $\text{sample}(p\_matrix, n\_samples, type, \beta)$ :
4       $samples \leftarrow \text{empty list}$ 
5      repeat
6           $mask \leftarrow \text{ones}(E)$ 
7           $center \leftarrow \text{zeros}(E)$ , hash map (default value is 0)
8           $sample \leftarrow \text{empty list}$ 
9          for  $i$  from 0 to  $\text{len}(p\_matrix) - 1$  do
10              $p \leftarrow \text{norm}(p\_matrix[i] * mask)$ 
11              $cls \leftarrow \text{sample a value from a multinomial}(1, E, p)$ 
12             if  $type$  is expert then
13                  $center[cls] \leftarrow center[cls] + 1$ 
14                 if  $center[cls] == N/E$  then
15                      $mask[cls] \leftarrow 0$ 
16             if  $type$  is token then
17                  $center[cls] \leftarrow center[cls] + \text{sum}(\mathcal{C}_p[i] * \mathbf{a}[i])$ 
18                 if  $center[cls] \geq \text{sum}(\mathcal{C}_p * \mathbf{a}) / E * \beta$  then
19                      $mask[cls] \leftarrow 0$ 
20              $append\ cls\ to\ sample$ 
21          $append\ sample\ to\ samples$ 
22     until generated  $n\_samples$  samples
23     return  $samples$ 
24
25 repeat
26      $ep\_samples \leftarrow \text{sample}(p\_matrix\_ep, K, expert)$ 
27      $tk\_samples \leftarrow \text{sample}(p\_matrix\_tk, K, token, f)$ 
28      $scores \leftarrow \text{summation of } \mathcal{T} \text{ of the same co-clusters}$ 
29      $better\_samples \leftarrow \text{samples with top-}\rho \times 100\% \text{ scores}$ 
30      $update\ p\_matrix\_ep\ and\ p\_matrix\_tk$ 
31 until iterating for  $n\_steps$  steps
32
33  $\mathcal{E} \leftarrow \text{argmax}(p\_matrix\_ep, axis=1)$ 
34  $\mathcal{T}, \mathcal{T}_p \leftarrow \text{argmax\_with\_values}(p\_matrix\_tk, axis=1)$ 

```

back l layers, we construct a table shaped like $[E^l, E]$, where the row of the table indicates the sequence of devices selected at the previous l layers and the column indicates the probability of activating the E devices in the current layer. Like the § B shows, we also calculate the activation sequence to device table $\mathcal{A}$ and the confidence table $\mathcal{A}_p$. In practice, we set the number of looking-back layers as 2.

B.2. Speculative Token Shuffling on the Fly Based on Fast Lookup

To reduce the combination space, we model the activation device sequence rather than the activation expert sequence, because we only care about the device-level token rebatching.

We implement a fast online re-batching mechanism based on fast looking-up tables (Algorithm 2). The algorithm queries the token-to-expert-cluster scheduling table $\mathcal{T}$ and expert-cluster-sequence-to-expert-cluster table $\mathcal{S}$, together with their confidences first. Then, the table with higher confidence is adopted to obtain the device ID list to which the current batch token needs to be shuffle (line 2). The algorithm performs the argsort operation to obtain the shuffle indicators (line 3) of the token. Then, the final shuffle indicators are obtained by grouping, aligning, and concatenation, and the token is shuffled (line 4 to line 7). After rebatching is complete, s-MoE calls the `reduce-scatter` operation. After MoE computing is complete, s-MoE runs the `allgather` operation to collect tokens. Finally, the order of tokens are shuffled back based on the previously calculated shuffle indicators (lines 14-18). The entire `rebatch` and `resume` processes do not involve

Algorithm 2 Online re-batching based on fast lookup

input: $\mathcal{B} \in \mathbb{N}^n$: Input token IDs; $\mathcal{T}$: token-to-expert-cluster Schedule Table; $\mathcal{A}$: expert-cluster-sequence-to-expert-cluster Schedule Table

35 **Function** `rebatch_tokens` ($\mathcal{B}, \mathcal{T}$):

```

36    $dev\_ids \leftarrow \text{cond}(\mathcal{T}_p[\mathcal{B}] > \mathcal{A}_p[\mathcal{B}], \mathcal{T}[\mathcal{B}], \mathcal{A}[\mathcal{B}])$ 
37    $shf\_indices \leftarrow \text{argsort}(dev\_ids)$ 
38    $g\_shf\_indices \leftarrow \text{group\_by\_key}(shf\_indices)$ 
39    $g\_shf\_indices \leftarrow \text{align}(g\_shf\_indices)$ 
40    $shf\_indices \leftarrow \text{concat}(g\_shf\_indices)$ 
41    $\mathcal{B} \leftarrow \mathcal{B}[shf\_indices]$ 
42   return  $shf\_indices$ 

```

43
44 **Function** `resume_tokens` ($\mathcal{B}, shf_indices$):

```

45    $r\_shf\_indices \leftarrow \text{argsort}(shf\_indices)$ 
46    $\mathcal{B} \leftarrow \mathcal{B}[r\_shf\_indices]$ 

```

```

47
48  $shf\_indices \leftarrow \text{rebatch\_tokens}(\mathcal{B}, \mathcal{T})$ 
49  $\mathcal{B}_{local} \leftarrow \text{reduce\_scatter}(\mathcal{B})$ 
50 executing MoE layer  $\mathcal{B} \leftarrow \text{allgather}(\mathcal{B}_{local})$ 
51 resume_tokens ( $\mathcal{B}, shf\_indices$ )

```

load calculation and decision-making. They are directly completed by querying tables. The runtime overhead mainly involves large token matrix shuffling, which we optimize via high-performance kernels. The memory occupation of the scheduling tables is negligible. For example, for DeepSeek-V2, the memory space that the token-to-device table $\mathcal{T}$ occupies is $\frac{102400 \times 60 \times 2}{1024^2} \approx 11.72MB$ (assuming the data format is `int16`).